CSE721: Encryption/Decryption Tool & Hill Cracker

Report generated: 2026-01-02 19:09:23

Overview

This project implements two deliverables for CSE721: (1) a multi-cipher encryption/decryption tool supporting Caesar, Affine, Playfair, and Hill (2x2) ciphers; and (2) a Hill cipher known-plaintext attack cracker. The implementation is written in C++ with console-based menus for clarity and reproducibility. The repository also provides placeholders for screenshots and an optional master launcher.

Repository & Layout

Repository: https://github.com/tareq056/CSE721-Crypto-Tool

Directory structure:
- src/part1_tool.cpp: Classic cipher tool (encryption/decryption for Caesar, Affine, Playfair, Hill 2x2).
- src/part2_hill_cracker.cpp: Known-plaintext attack (Hill 2x2) cracker.
- src/main.cpp: Optional master menu wrapper.
- Final Part 01.cpp / Final Part 02.cpp: Original full solutions retained.
- screenshots/: place run-time captures.
- README.md: quick start.
- REPORT.pdf: this document.

Architecture & Design

Part 1 (Classic Cipher Tool): Uses modular helpers for text normalization, modular arithmetic, and matrix ops (2x2). Each cipher has a dedicated function pair for encrypt/decrypt with menu-driven selection. The Hill cipher uses row-vector form with key validation (determinant invertibility mod 26). Playfair preprocessing handles digraph formation, X-padding, and J/I handling. Input is sanitized to uppercase alphabetic content to keep math predictable.

Part 2 (Hill Cracker): Implements a known-plaintext attack on Hill 2x2. Users provide matching plaintext/ciphertext pairs; the tool solves for the key matrix using modular inverses (determinant must be coprime with 26). Once a key is recovered, it decrypts remaining ciphertext. Defensive checks ensure valid key recovery and graceful failure when input is insufficient.

Common helpers: mod26 arithmetic, extended Euclid for modular inverses, string normalization to letters-only uppercase, digraph packing/unpacking, and matrix inversion for 2x2 mod 26.

Dependencies

Language: C++17 or later. Standard library only (iostream, string, vector, cctype, limits, utility). No external libraries required to build/run.

Build & Run

Example commands (Windows, MinGW/Clang assumed; adjust compiler as needed):

- Build Part 1 tool:

```
g++ -std=c++17 -O2 src/part1_tool.cpp -o part1_tool.exe
```

- Build Part 2 cracker:

CSE721: Encryption/Decryption Tool & Hill Cracker

Report generated: 2026-01-02 19:09:23

```
g++ -std=c++17 -O2 src/part2_hill_cracker.cpp -o hill_cracker.exe
```

- Optional master menu:

```
g++ -std=c++17 -O2 src/main.cpp -o crypto_tool.exe
```

Run the resulting .exe from a console to access the menus. For Playfair/Hill inputs, enter alphabetic characters; the tool normalizes casing and strips spaces/punctuation.

Cipher Operations (Part 1)

Caesar: Shift by k (0-25) with wrap-around. Supports encryption and decryption.

Affine: E(x)= (a*x + b) mod 26 with user-provided (a,b). Validates gcd(a,26)=1; computes modular inverse for decryption.

Playfair: Builds 5x5 key square (I/J merge), preprocesses plaintext into digraphs with X padding, handles repeated-letter digraph splitting. Implements standard Playfair rules for same-row, same-column, and rectangle cases.

Hill 2x2: Uses row-vector form P*K mod 26. Validates key determinant invertibility mod 26; computes inverse key for decryption. Input is grouped into digraphs; padding added if needed to keep even length.

Known-Plaintext Attack (Part 2)

Goal: recover 2x2 Hill key matrix K from plaintext/ciphertext digraph pairs and decrypt ciphertext.
Workflow:
1) Normalize plaintext/ciphertext to uppercase letters; split into digraphs.
2) Take first two plaintext digraphs as rows of matrix P, first two ciphertext digraphs as rows of matrix C.
3) Compute K = P^{-1} * C (mod 26); requires det(P) coprime with 26.
4) If inversion fails, request different digraph pairs.
5) Use recovered K to decrypt remaining ciphertext and display key and plaintext.
The tool surfaces validation errors early to avoid silent wrong keys.

Usage Notes & UX

- Menu-driven console prompts for cipher selection and mode (encrypt/decrypt).
- Input sanitation removes non-letters and uppercases to keep math consistent.
- Key validation: Affine checks gcd(a,26)=1; Hill checks determinant invertible; Playfair merges I/J and pads with X.
- Output echoes normalized text so users can verify preprocessing.
- Errors are handled with friendly messages and re-prompts where applicable.

Testing Performed

- Caesar/Affine spot checks with small shifts to confirm inverse correctness.
- Playfair tested with repeated letters and odd-length inputs to verify padding and rectangle logic.

CSE721: Encryption/Decryption Tool & Hill Cracker

Report generated: 2026-01-02 19:09:23

- Hill 2x2 tested with known invertible keys; decryption confirmed against encryption.
- Hill cracker validated by encrypting sample plaintext with a known key and recovering it via the tool, then decrypting the ciphertext to the original plaintext.

Screenshots

Place run-time captures (encryption, decryption, Hill cracking) in the screenshots/ directory. Filenames suggested: part1_encrypt.png, part1_decrypt.png, hill_cracker.png.

Credits & Tools Used

Developed in C++ using standard library only. This report generated via Python fpdf2. LLM assistance (ChatGPT) used for scaffolding and documentation drafting; code and logic verified manually.

Next Steps

1) Integrate src/main.cpp to dispatch between Part 1 and Part 2 binaries.
2) Add automated test vectors (e.g., small Catch2 or GoogleTest harness) for regressions.
3) Bundle Windows build scripts for one-click compilation.
4) Record demo GIFs for README and screenshots/.